

ОПТИМИЗАЦИЯ

Оптимизация SQL-запросов — это выбор наилучшего **плана выполнения**, минимизирующего ресурсы (I/O, CPU, память). В PostgreSQL она реализуется через несколько этапов: парсинг, преобразование, планирование и исполнение. СУБД учитывает законы реляционной алгебры, статистику таблиц и наличие индексов. Хорошая оптимизация = меньше чтения, меньше записей, больше конвейерной обработки.

Общий процесс выполнения запроса в PostgreSQL

1. **Парсинг (parser):**
Синтаксический разбор → дерево запроса.
2. **Переписывание (rewriter):**
Обработка представлений, правил, привилегий.
3. **Планировщик и оптимизатор (planner):**
 - Построение **всех возможных планов** выполнения.
 - Выбор **наиболее эффективного** (по стоимости).
4. **Исполнение (executor):**
 - Пошаговое выполнение плана — с JOIN, фильтрацией, сортировкой и т.д.

Как оптимизатор выбирает план

Основной критерий: оценочная стоимость плана

Она учитывает:

- 📦 **Число обращений к внешней памяти;**
- 🕒 **Время чтения/записи;**
- 💾 **Размеры промежуточных данных.**

Чем меньше «стоимость» — тем лучше.

Законы, которые применяет оптимизатор (реляционная алгебра)

1. **Коммутативность соединений**
 $R \bowtie S \equiv S \bowtie R$
2. **Ассоциативность**
 $R \bowtie (S \bowtie T) \equiv (R \bowtie S) \bowtie T$
3. **Выборка может быть вынесена "вниз"**
 $\sigma_f(R \bowtie S) \equiv \sigma_f(R) \bowtie S$ (если f касается только R)
4. **Двойная выборка = вложенные фильтры**
 $\sigma_{\theta \wedge f}(R) \equiv \sigma_{\theta}(\sigma_f(R))$
5. **Проекция может быть применена к R до соединения**
 $\pi_A(R \bowtie S) \equiv \pi_A(\pi_{A \cup B}(R) \bowtie S)$

Стратегии оптимизации

- Делать выборку раньше (сначала $\sigma_{A.type='X'}(A)$, потом JOIN)
 - `SELECT * FROM A JOIN B ON A.id = B.aid WHERE A.type = 'X';`
- Делать проекцию раньше (убрать ненужные поля до соединения)
 - `SELECT name FROM STUDENTS JOIN GROUPS ON ...;`
- Использовать **левосторонние деревья, планы с левой ассоциативностью**:
 - позволяют избежать **материализации**
 - можно обрабатывать **конвейером**

Конвейерная обработка vs материализация

Конвейер

- Результат одной операции **сразу** подается следующей
- Минимум памяти, минимум времени

Материализация

- Сохраняются промежуточные результаты
- Затраты на **запись/чтение**
- Может потребоваться при DISTINCT, GROUP BY, ORDER BY

Виды JOIN-ов и их стоимость

1. Nested Loop Join

- Простая реализация
- Хорошо при малом числе строк (особенно во внутренней таблице)
- Плохо, когда обе таблицы большие и без индекса

для каждой строки r из R :
 для каждой строки s из S :
 если r и s удовлетворяют условию соединения:
 вернуть $r + s$

2. Hash Join

- Создает хеш-таблицу по одной таблице (в памяти)
- Идеален для = соединений, плох в остальном
- Операция требует много памяти

Создать хеш-таблицу по S : $key \rightarrow$ список строк
 для каждой строки r из R :
 найти в хеш-таблице строки s , где $s.key = r.key$
 для каждой найденной пары (r, s) :
 вернуть $r + s$

3. Sort-Merge Join

- Сортирует обе таблицы $\rightarrow$ сливает
- Лучше при уже отсортированных данных или больших таблицах
- Требуется **материализацию**
- Требуется сортировка на лету (если нет индекса или кластеризации)

Отсортировать R по key
 Отсортировать S по key
 Пока не достигнут конец обоих списков:
 сравнить $r.key$ и $s.key$
 если равны – вернуть комбинацию (может быть несколько вариантов)
 иначе – продвинуться по меньшему

Индексы в оптимизации

Индексы особенно полезны:

- Для WHERE, JOIN, ORDER BY, GROUP BY
- При ограниченных выборках

Но: могут ухудшить производительность, если:

- выборка большая
- данные не селективны
- таблица маленькая

Планировщик может **не использовать** индекс, даже если он есть!

JOIN тип	Поддержка =	Подходит для больших таблиц	Требуется индексов	Использует сортировку	Использует хеш
Nested Loop	✓	✗ (если без индекса)	✓ желательно	✗	✗
Hash Join	✓	✓	✗	✗	✓
Sort-Merge Join	✓ /диапазон	✓	✗ (но желательно)	✓	✗